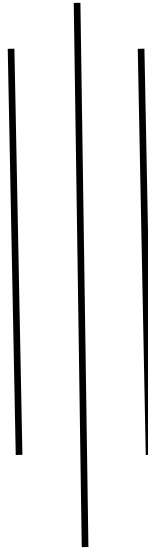


Madan Bhandari Memorial College
New Baneshwor, Kathmandu



Lab Report of Advance Java Programming

Submitted by:

Name: Srijana Subedi

Code: 3279616

Faculty: Humanities and Social Science

Semester: Sixth

Submitted to :

Department of BCA

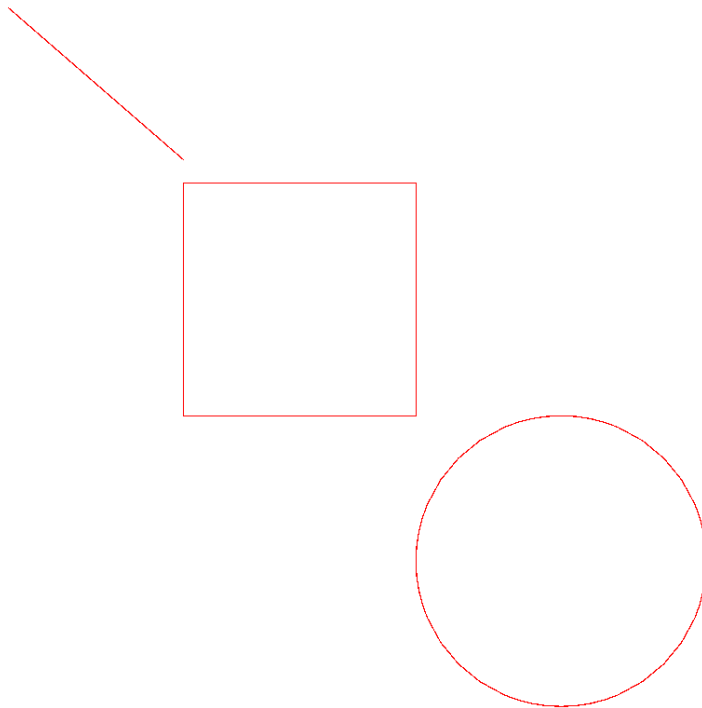
Table Of Content

S.N	Title	Date	Signature
1	Write a Program to draw line, rectangle and ellipse using graphics 2D in swing.		
2	Write a program to draw a rectangle and ellipse and fill different color on it.		
3	Write a program to show the menu bar .		
4	Write a program to show the function of layout manager.		
5	Write a program to design a form having text box as name ,age ,course ,contact and address. Also add submit button and when user click the submit the details is shown in the next page.		
6	Write a program to design a calculator where user provide two number to add ,subtract ,multiply and divide .Add all validation and show the result in message box.		
7	Write a program to show the jdbc connection and perform crud operation.		
8	Write a progarm to show the MVC pattern.		
9	Write a client server application in RMI to find the greatest number among three input from the user .		
10	Create a java RMI application where the client sends a number to the server and the server returns the square of that number .		
11	Develop a web-based student portal application using Java Servlets and JSP that demonstrates the complete servlet lifecycle, effectively handles HTTP requests and responses, and utilizes both javax.servlet and javax.servlet.http packages. The application should process user input through HTML forms, manage user sessions using cookies and session tracking mechanisms, and display dynamic content with JSP pages.		
12	Design and Implement java beans with its properties.		

Write a Program to draw line, rectangle and ellipse using graphics 2D in swing.

Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.geom.Ellipse2D;
class Shape extends JFrame {
    public Shape() {
        super("Rectangles Drawing Demo");
        getContentPane().setBackground(Color.WHITE);
        setSize(480, 200);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
    }
    void drawShapes(Graphics g) {
        Graphics2D g2d = (Graphics2D) g;
        g2d.setColor(Color.red);
        g2d.drawRect(200, 200, 200, 200);
        g2d.drawLine(50, 50, 200, 180);
        g2d.draw(new Ellipse2D.Double(400, 400, 250, 250));
    }
    public void paint(Graphics g) {
        super.paint(g);
        drawShapes(g);
    }
    public static void main(String[] args) throws Exception {
        SwingUtilities.invokeLater(new Runnable() {
            @Override
            public void run() {
                new Shape().setVisible(true);
            }
        });
    }
}
```



Write a program to draw a rectangle and ellipse and fill different color on it.

Code:

```
package org.example;

import javax.swing.*.*;
import java.awt.*.*;

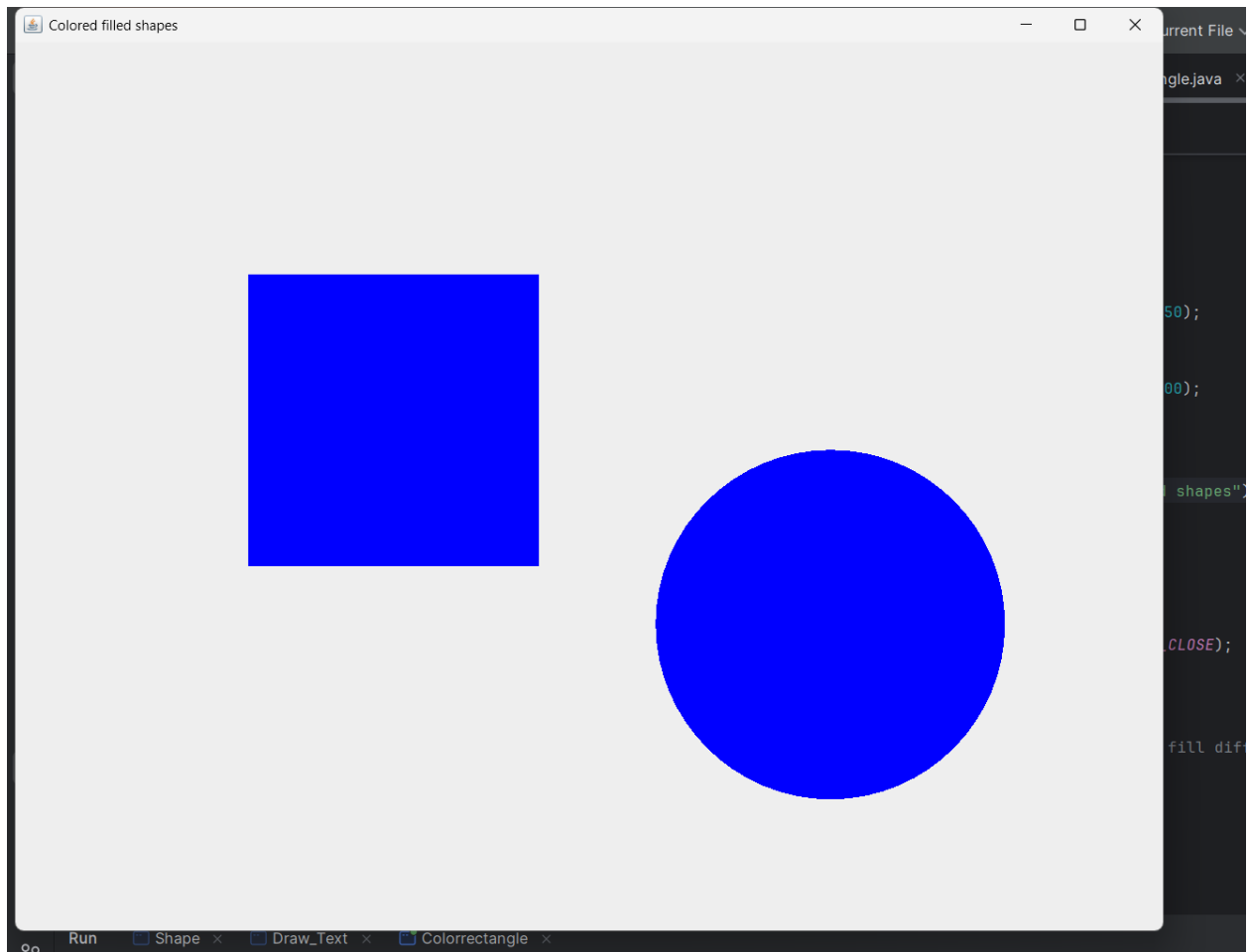
public class Colorrectangle extends JPanel {

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        g.setColor(Color.BLUE);
        g.fillRect(200, 200, 250, 250);
        g.fillOval(550, 350, 300, 300);
    }

    public static void main(String[] args) {
        JFrame frame = new JFrame("Colored filled shapes");

        Colorrectangle panel = new Colorrectangle();
        frame.add(panel);
    }
}
```

```
frame.setSize(1000, 800);  
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
frame.setVisible(true);  
}  
}
```



Write a program to show the menu bar .

Code:

```
package org.example;
import javax.swing.*;
import java.awt.event.*;
class menu {
    public static JMenuBarcreateMenuBar() {
JMenuBarmenuBar;
JMenu menu;
JMenuItemmenuItem;
JCheckBoxMenuItemcbmi;
JRadioButtonMenuItemrbmi;
menuBar = new JMenuBar();
        menu = new JMenu("File");
menu.setMnemonic(KeyEvent.VK_F);
menuBar.add(menu);
menuItem = new JMenuItem("New Project");
menuItem.setAccelerator(KeyStroke.getKeyStroke(
KeyEvent.VK_N, ActionEvent.CTRL_MASK));
menu.add(menuItem);
menuItem = new JMenuItem("Open");
menuItem.setAccelerator(KeyStroke.getKeyStroke(
KeyEvent.VK_O, ActionEvent.CTRL_MASK));
menu.add(menuItem);
menuItem = new JMenuItem("Exit");
menuItem.addActionListener(e ->System.exit(0));
menu.addSeparator();
menu.add(menuItem);
        menu = new JMenu("Edit");
menu.setMnemonic(KeyEvent.VK_E);
menuBar.add(menu);
menuItem = new JMenuItem("Cut");
menu.add(menuItem);
menuItem = new JMenuItem("Copy");
menu.add(menuItem);
menuItem = new JMenuItem("Paste");
menu.add(menuItem);
        menu = new JMenu("View");
menu.setMnemonic(KeyEvent.VK_V);
menuBar.add(menu);
cbmi = new JCheckBoxMenuItem("Show Toolbar");
cbmi.setSelected(true);
menu.add(cbmi);
cbmi = new JCheckBoxMenuItem("Show Status Bar");
cbmi.setSelected(true);
menu.add(cbmi);

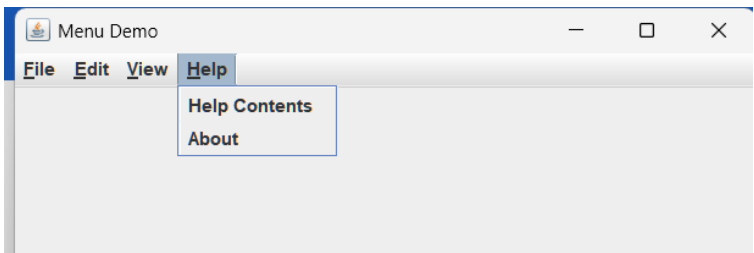
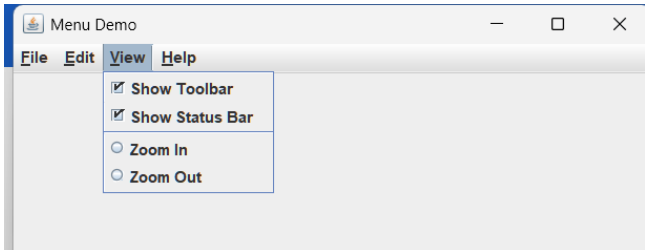
menu.addSeparator();
ButtonGroupbg = new ButtonGroup();
rbmi = new JRadioButtonMenuItem("Zoom In");
```

```

bg.add(rbmi);
menu.add(rbmi);
rbmi = new JRadioButtonMenuItem("Zoom Out");
bg.add(rbmi);
menu.add(rbmi);
    menu = new JMenu("Help");
menu.setMnemonic(KeyEvent.VK_H);
menuBar.add(menu);
menuItem = new JMenuItem("Help Contents");
menu.add(menuItem);
menuItem = new JMenuItem("About");
menuItem.addActionListener(e ->
JOptionPane.showMessageDialog(null,
    "Menu Demo Application\nVersion 1.0",
    "About",
JOptionPane.INFORMATION_MESSAGE));
menu.add(menuItem);
    return menuBar;
}
public static void main(String[] args) {
JFrame frame = new JFrame("Menu Demo");
frame.setJMenuBar(createMenuBar());
frame.setSize(500, 300);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
frame.setVisible(true);
}
}

```

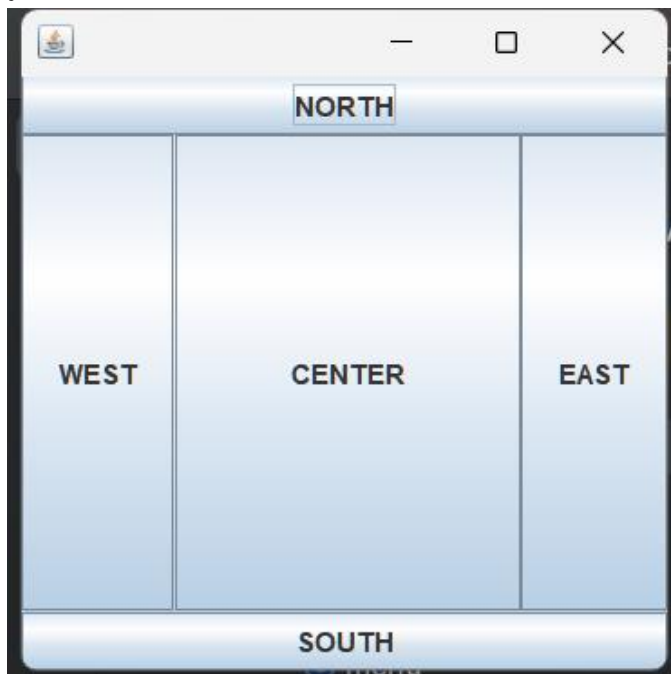




Write a program to show the function of layout manager.

Code:

```
package org.example;
import java.awt.*;
import javax.swing.*;
class Border{
    JFrame f;
    Border() {
        f = new JFrame();
        JButton b1 = new JButton("NORTH");
        JButton b2 = new JButton("SOUTH");
        JButton b3 = new JButton("EAST");
        JButton b4 = new JButton("WEST");
        JButton b5 = new JButton("CENTER");
        f.add(b1, BorderLayout.NORTH);
        f.add(b2, BorderLayout.SOUTH);
        f.add(b3, BorderLayout.EAST);
        f.add(b4, BorderLayout.WEST);
        f.add(b5, BorderLayout.CENTER);
        f.setSize(300, 300);
        f.setVisible(true); }
    public static void main(String[] args) {
        new Border();
    }
}
```



Write a program to design a form having text box as name ,age ,course ,contact and address. Also add submit button and when user click the submit the details is shown in the next page.

Code:

```
package org.example;
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
class StudentForm extends JFrame implements ActionListener {
JLabel l1, l2, l3, l4, l5;
JTextField t1, t2, t3, t4;
JTextArea ta;
JButton submit;
StudentForm() {
setTitle("Student Registration Form");
setSize(400, 420);
setLocationRelativeTo(null);
setDefaultCloseOperation(EXIT_ON_CLOSE);
setLayout(null);
    l1 = new JLabel("Name:");
    l1.setBounds(30, 30, 100, 25);
    add(l1);
    t1 = new JTextField();
    t1.setBounds(150, 30, 180, 25);
    add(t1);
    l2 = new JLabel("Age:");
    l2.setBounds(30, 70, 100, 25);
    add(l2);
    t2 = new JTextField();
    t2.setBounds(150, 70, 180, 25);
    add(t2);
    l3 = new JLabel("Course:");
    l3.setBounds(30, 110, 100, 25);
    add(l3);
    t3 = new JTextField();
    t3.setBounds(150, 110, 180, 25);
    add(t3);
    l4 = new JLabel("Contact:");
    l4.setBounds(30, 150, 100, 25);
    add(l4);
    t4 = new JTextField();
    t4.setBounds(150, 150, 180, 25);
    add(t4);
    l5 = new JLabel("Address:");
    l5.setBounds(30, 190, 100, 25);
    add(l5);
    ta = new JTextArea();
    ta.setBounds(150, 190, 180, 60);
    add(ta);
}
```

```

        submit = new JButton("Submit");
submit.setBounds(150, 280, 100, 30);
        add(submit);
submit.addActionListener(this);
setVisible(true); }
@Override
public void actionPerformed(ActionEvent e) {
    String name = t1.getText().trim();
    String ageText = t2.getText().trim();
    String course = t3.getText().trim();
    String contact = t4.getText().trim();
    String address = ta.getText().trim();
    if (name.isEmpty()) {
JOptionPane.showMessageDialog(this, "Name is required");
        return; }
    if (ageText.isEmpty()) {
JOptionPane.showMessageDialog(this, "Age is required");
        return; }
    int age;
    try { age = Integer.parseInt(ageText);
        if (age <= 0) {
JOptionPane.showMessageDialog(this, "Age must be greater than 0");
            return; }
        } catch (NumberFormatException ex) {
JOptionPane.showMessageDialog(this, "Age must be a number");
            return; }
    if (course.isEmpty()) {
JOptionPane.showMessageDialog(this, "Course is required");
        return; }
    if (contact.isEmpty()) {
JOptionPane.showMessageDialog(this, "Contact number is required");
        return; }
    if (!contact.matches("\\d{10}")) {
JOptionPane.showMessageDialog(this, "Contact number must be 10 digits");
        return; }
    if (address.isEmpty()) {
JOptionPane.showMessageDialog(this, "Address is required");
        return;
    }
    new DisplayPage(name, ageText, course, contact, address);
    dispose();
}
public static void main(String[] args) {
    new StudentForm();
}
}

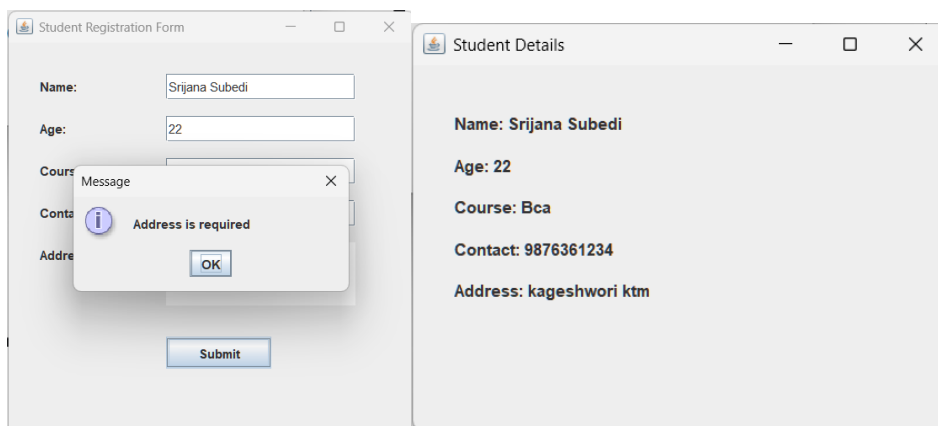
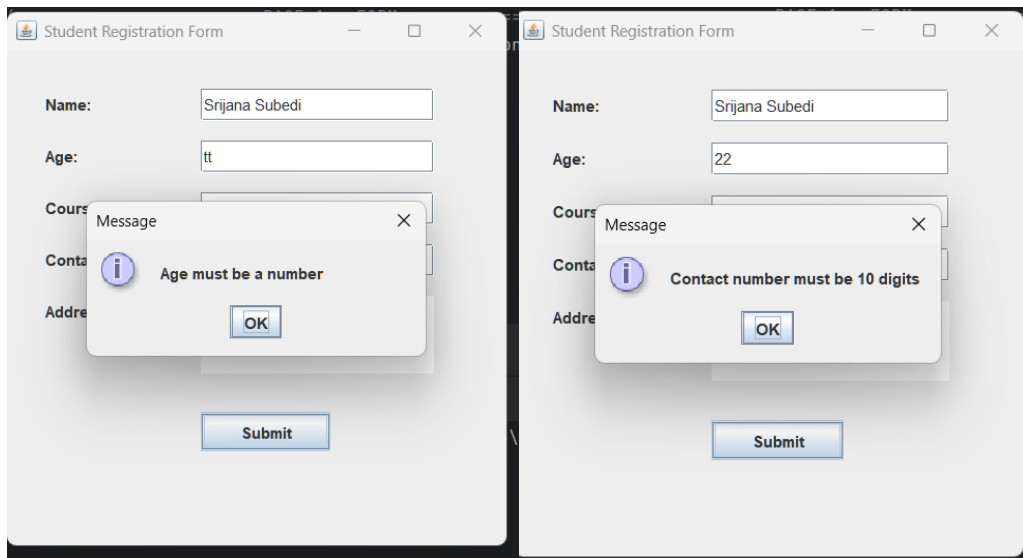
class DisplayPage extends JFrame {
DisplayPage(String name, String age, String course, String contact, String address) {
setTitle("Student Details");
setSize(400, 300);
setLocationRelativeTo(null);
setLayout(null);

```

```

JLabel l1 = new JLabel("Name: " + name);
l1.setBounds(30, 30, 300, 25);
add(l1);
JLabel l2 = new JLabel("Age: " + age);
l2.setBounds(30, 60, 300, 25);
add(l2);
JLabel l3 = new JLabel("Course: " + course);
l3.setBounds(30, 90, 300, 25);
add(l3);
JLabel l4 = new JLabel("Contact: " + contact);
l4.setBounds(30, 120, 300, 25);
add(l4);
JLabel l5 = new JLabel("Address: " + address);
l5.setBounds(30, 150, 300, 25);
add(l5);
setVisible(true);
}
}

```



Write a program to design a calculator where user provide two number to add ,subtract ,multiply and divide .Add all validation and show the result in message box.

Code:

```
package org.example;
import javax.swing.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
public class calculator extends JFrame implements ActionListener {
    JLabel l1, l2;
    JTextField t1, t2;
    JButton add, sub, mul, div;
    calculator() {
        setTitle("Simple Calculator");
        setSize(350, 250);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(null);
        l1 = new JLabel("First Number:");
        l1.setBounds(30, 30, 100, 25);
        add(l1);
        t1 = new JTextField();
        t1.setBounds(150, 30, 120, 25);
        add(t1);
        l2 = new JLabel("Second Number:");
        l2.setBounds(30, 70, 100, 25);
        add(l2);
        t2 = new JTextField();
        t2.setBounds(150, 70, 120, 25);
        add(t2);
        add = new JButton("Add");
        add.setBounds(30, 120, 70, 30);
        add(add);
        sub = new JButton("Subtract");
        sub.setBounds(110, 120, 90, 30);
        add(sub);
        mul = new JButton("Multiply");
        mul.setBounds(210, 120, 90, 30);
        add(mul);
        div = new JButton("Divide");
        div.setBounds(110, 160, 90, 30);
        add(div);
        add.addActionListener(this);
        sub.addActionListener(this);
        mul.addActionListener(this);
        div.addActionListener(this);
        setVisible(true);
    }

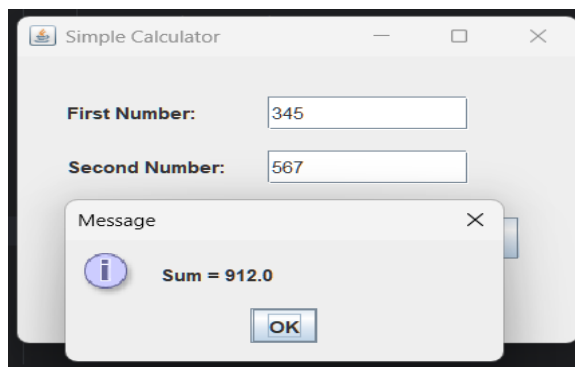
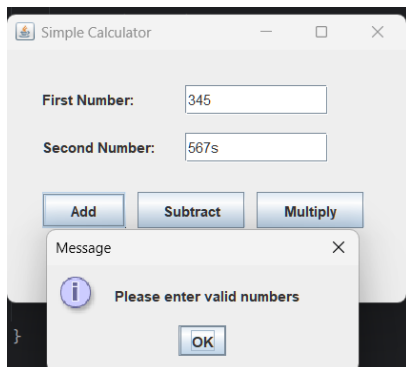
    @Override
    public void actionPerformed(ActionEvent e) {
```

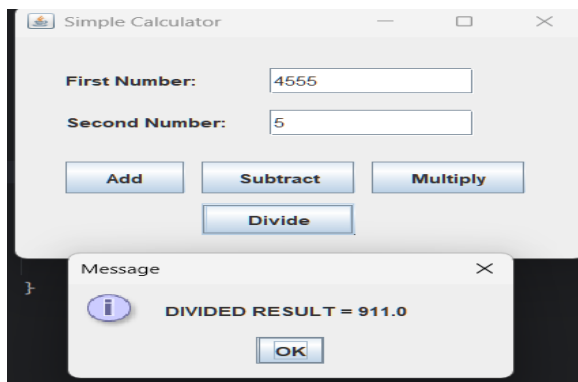
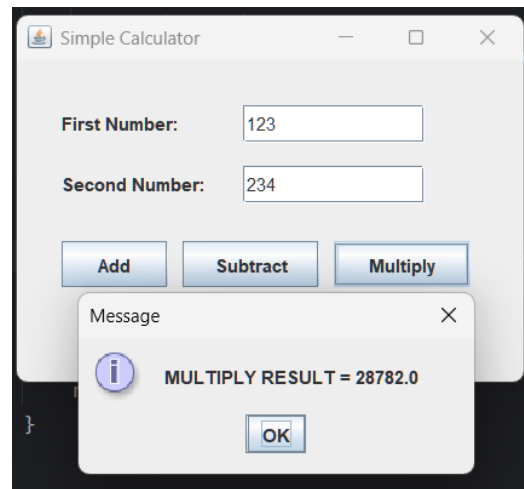
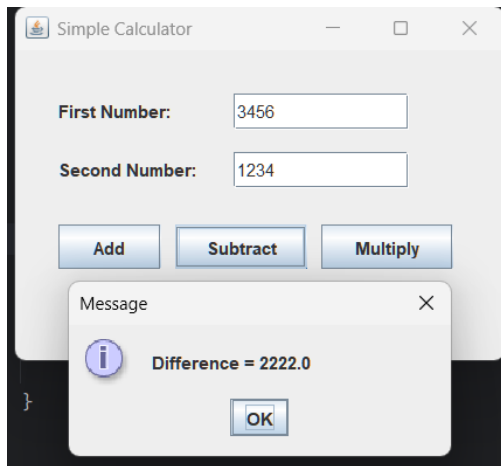
```

        if (t1.getText().isEmpty() || t2.getText().isEmpty()) {
JOptionPane.showMessageDialog(this, "Please enter both numbers");
            return;
        }
        double a, b;
        try {
            a = Double.parseDouble(t1.getText());
            b = Double.parseDouble(t2.getText());
        } catch (NumberFormatException ex) {
JOptionPane.showMessageDialog(this, "Please enter valid numbers");
            return;
        }
        double result = 0;
        if (e.getSource() == add) {
            result = a + b;
        } else if (e.getSource() == sub) {
            result = a - b;
        } else if (e.getSource() == mul) {
            result = a * b;
        } else if (e.getSource() == div) {
            if (b == 0) {
JOptionPane.showMessageDialog(this, "Cannot divide by zero");
                return;
            }
            result = a / b;
        }
JOptionPane.showMessageDialog(this, "RESULT = " + result);
    }

    public static void main(String[] args) {
        new calculator();
    }
}

```





Write a program to show the MVC pattern.

Model (Student .java)

```
© Student.java ×  © controller\StudentController.java
1  package model;
2
3  public class Student { 11 usages
4      private int id; 2 usages
5      private String name; 2 usages
6      private String address; 2 usages
7      private String phone; 2 usages
8      private String email; 2 usages
9
10     public Student(int id, String name, String address, String phone, String email) { 4 usages
11         this.id = id;
12         this.name = name;
13         this.address = address;
14         this.phone = phone;
15         this.email = email;
16     }
17
18     // Getters
19     public int getId() { return id; } 1 usage
20     public String getName() { return name; } 1 usage
21     public String getAddress() { return address; } 1 usage
22     public String getPhone() { return phone; } 1 usage
23     public String getEmail() { return email; } 1 usage
24 }
25
```

Model(StudentModel.java)

```
© Student.java  © StudentModel.java ×  © controller\StudentController.java
1  package model;
2
3  import java.util.ArrayList;
4  import java.util.List;
5
6  public class StudentModel { 6 usages
7
8      public List<Student> getAllStudents() { 1 usage
9          List<Student> students = new ArrayList<>();
10
11          // Hard-coded students
12          students.add(new Student(1, name: "Srijana Subedi", address: "Kathmandu", phone: "98784533672", email: "sri-
13          students.add(new Student(2, name: "Sita", address: "Pokhara", phone: "9811111111", email: "sita@gmail.com"));
14          students.add(new Student(3, name: "sajaan", address: "Lalitpur", phone: "9822222222", email: "saajan@gmail-
15          students.add(new Student(4, name: "santosh", address: "Bhaktapur", phone: "9833333333", email: "santoosh@gm-
16
17          return students;
18      }
19  }
20
```

View(StudentView.java)

```
© Student.java  © StudentModel.java  © StudentView.java ×  © controller\StudentController.java
1  package view;
2
3  import model.Student;
4  import java.util.List;
5
6  public class StudentView { 6 usages
7
8  @ public void printStudentlist(List<Student> students) { 1 usage
9      System.out.println("ID | Name | Address | Phone | Email");
10     System.out.println("-----");
11     for (Student s : students) {
12         System.out.println(
13             s.getId() + " | " +
14             s.getName() + " | " +
15             s.getAddress() + " | " +
16             s.getPhone() + " | " +
17             s.getEmail()
18         );
19     }
20 }
21
22
```

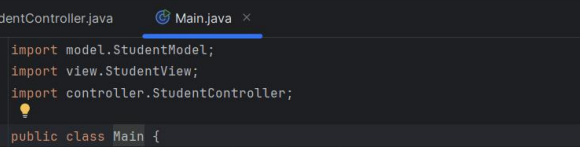

Controller(StudentController.java)

```

1 package controller;
2
3 import model.StudentModel;
4 import model.Student;
5 import view.StudentView;
6
7 import java.util.List;
8
9 public class StudentController { 3 usages
10     private StudentModel model; 2 usages
11     private StudentView view; 2 usages
12
13     public StudentController(StudentModel model, StudentView view) { 1 usage
14         this.model = model;
15         this.view = view;
16     }
17
18     public void displayAllStudents() { 1 usage
19         List<Student> students = model.getAllStudents();
20         view.printStudentList(students);
21     }
22 }
23

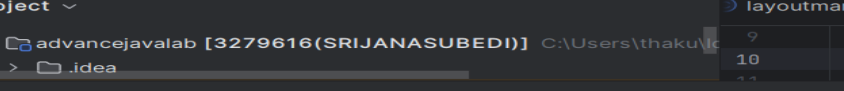
```

Main.java



```
Current File ▾ ▶ 🔍 ⋮
@ StudentController.java  @ Main.java ✕
1  import model.StudentModel;
2  import view.StudentView;
3  import controller.StudentController;
4  ⚡
5  ▶ public class Main {
6  ▶     public static void main(String[] args) {
7      // Initialize MVC components
8      StudentModel model = new StudentModel();
9      StudentView view = new StudentView();
10     StudentController controller = new StudentController(model, view);
11
12     // Display all students
13     controller.displayAllStudents();
14 }
15
16     StudentController controller = new StudentController(model, view)
    3279616(SRIJANASUBEDI) 🔗 ⋮
```

Output



```
Project > layoutmanage.java
  > advancejavalab [3279616(SRIJANASUBEDI)] C:\Users\thakur\Idea
    > .idea
Run Main x
C:\Program Files\Java\jdk-21\bin\java.exe" "-javaagent:C:\Program Fi
ID | Name | Address | Phone | Email
-----
1 | Srijana Subedi | Kathmandu | 98784533672 | srij@gmail.com
2 | Sita | Pokhara | 9811111111 | sita@gmail.com
3 | sajaan | Lalitpur | 9822222222 | saajan@gmail.com
4 | santosh | Bhaktapur | 9833333333 | santoosh@gmail.com
Process finished with exit code 0
```

Write a client server application in RMI to find the greatest number among three input from the user .

Code

GreatestClient.java

```
package org.example;

import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.util.Scanner;
class GreatestClient {
    public static void main(String[] args) {
        try {
            Registry registry = LocateRegistry.getRegistry("localhost", 1099);
            GreatestInterface stub =
                (GreatestInterface) registry.lookup("GreatestService");
            Scanner sc = new Scanner(System.in);
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();
            System.out.print("Enter third number: ");
            int c = sc.nextInt();
            int result = stub.findGreatest(a, b, c);
            System.out.println("Greatest number is: " + result);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

GreatestImplementation.java

```
package org.example;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
class GreatestImplementation extends UnicastRemoteObject implements GreatestInterface {
    protected GreatestImplementation() throws RemoteException { super();
    @Override
    public int findGreatest(int a, int b, int c) throws RemoteException {
        int max = a;
        if (b > max) max = b;
        if (c > max) max = c;
        return max;
    }
}
```

GreatestServer.java

```
package org.example;
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
class GreatestServer {
    public static void main(String[] args) {
```

```

    try {
        GreatestInterface obj = new GreatestImplementation();
        Registry registry = LocateRegistry.createRegistry(1099);
        registry.rebind("GreatestService", obj);
        System.out.println("RMI Server started...");
    } catch (Exception e) {
        e.printStackTrace();
    }
}
}

```

GreatestInterface.java

```

package org.example;

import java.rmi.Remote;

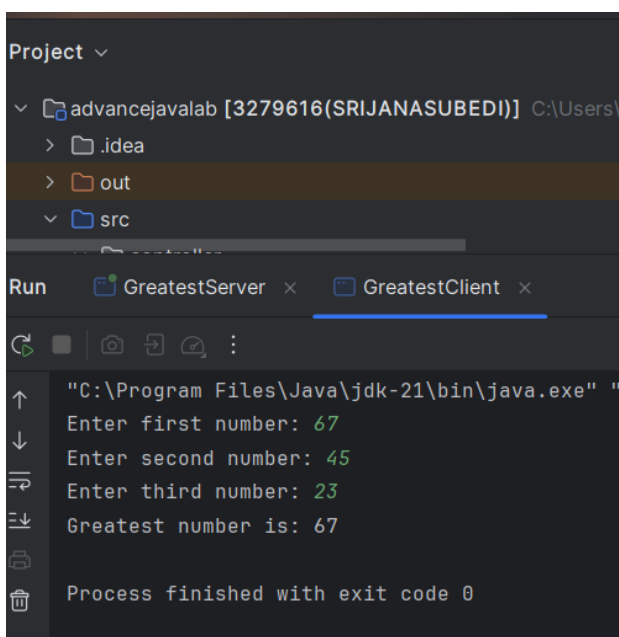
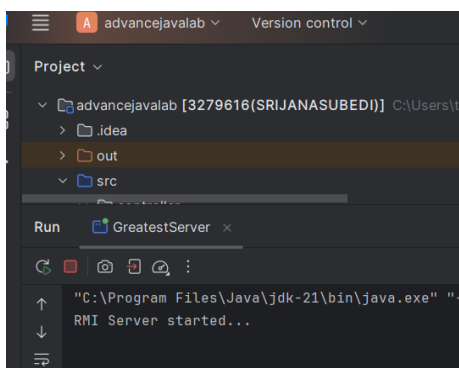
import java.rmi.RemoteException;

public interface GreatestInterface extends Remote {

    int findGreatest(int a, int b, int c) throws RemoteException;

}

```



Create a java RMI application where the client sends a number to the server and the server returns the square of that number .

Square.java

```
package rmi;

import java.rmi.*;
import java.rmi.RemoteException;

public interface Square extends Remote {
    int square(int n) throws RemoteException;
}
```

SquareImpl.java

```
package rmi;
import java.rmi.*;
import java.rmi.server.*;
public class SquareImpl extends UnicastRemoteObject implements Square {
    SquareImpl() throws RemoteException {
        super();
    }
    public int square(int n) throws RemoteException {
        return n * n;
    }
}
```

Server.java

```
package rmi;
import java.rmi.*;
import java.rmi.registry.*;
public class Server {
    public static void main(String args[]) {
        try {
            System.setProperty("java.rmi.server.hostname", "localhost");
            LocateRegistry.createRegistry(1099);
            SquareImpl obj = new SquareImpl();
            Naming.rebind("rmi://localhost/SquareService", obj);
            System.out.println("Server started...");
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

Client.java

```
package rmi;

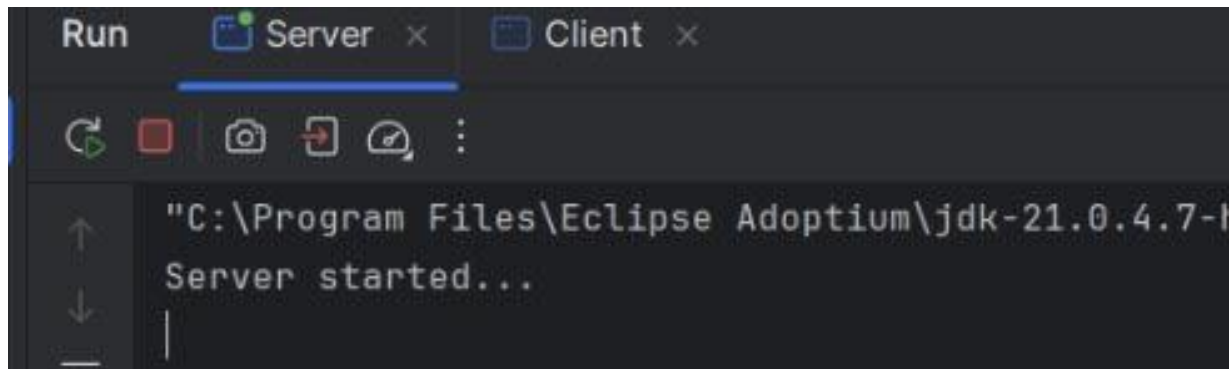
import java.rmi.*;

public class Client {
    public static void main(String args[]) {
        try {
            Square obj = (Square) Naming.lookup("rmi://localhost/SquareService");

            int result = obj.square(5);

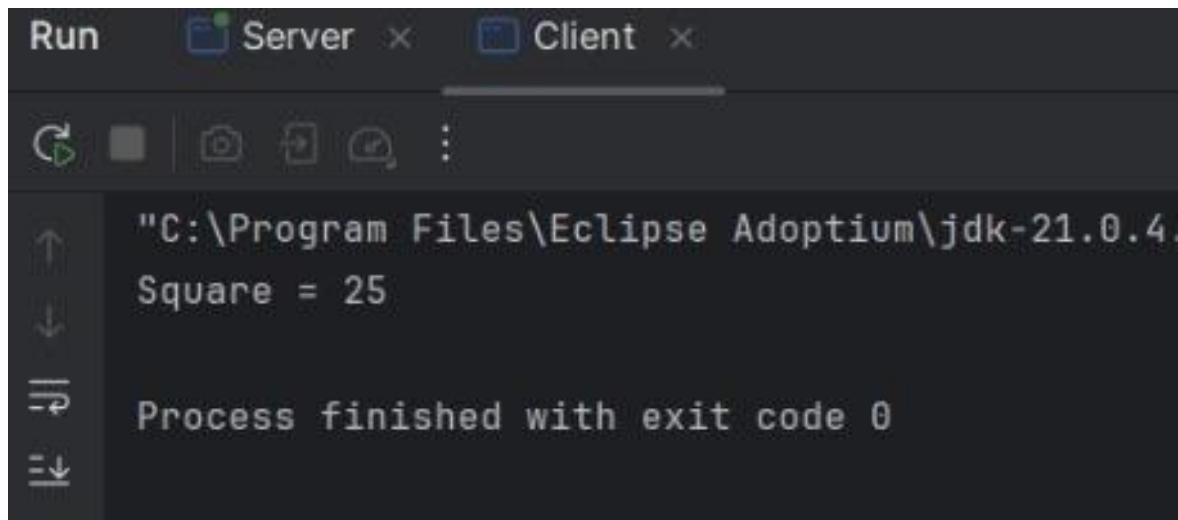
            System.out.println("Square = " + result);
        }
    }
}
```

```
    } catch(Exception e) {  
        System.out.println(e);  
    }  
}  
}
```



This screenshot shows the Eclipse IDE's Run console. The 'Server' tab is selected. The console output displays the Java path and the message 'Server started...'. The 'Client' tab is also visible but not active.

```
Run  Server x Client x  
"C:\Program Files\Eclipse Adoptium\jdk-21.0.4.7-1  
Server started...  
|
```



This screenshot shows the Eclipse IDE's Run console with the 'Client' tab selected. The console output shows the Java path, the result 'Square = 25', and the message 'Process finished with exit code 0'. The 'Server' tab is also visible but not active.

```
Run  Server x Client x  
"C:\Program Files\Eclipse Adoptium\jdk-21.0.4.  
Square = 25  
  
Process finished with exit code 0
```

Develop a web-based student portal application using Java Servlets and JSP that demonstrates the complete servlet lifecycle, effectively handles HTTP requests and responses, and utilizes both javax.servlet and javax.servlet.http packages. The application should process user input through HTML forms, manage user sessions using cookies and session tracking mechanisms, and display dynamic content with JSP pages.

Code:

index.html

```
<!DOCTYPE html>
<html>
<head><title>Student Login</title></head>
<body>
  <h2>Login</h2>
  <form action="login" method="post">
    <label>Username: <input type="text" name="username" required></label><br>
    <label>Password: <input type="password" name="password" required></label><br>
    <button type="submit">Login</button>
  </form>
</body>
</html>
```

LoginServlet.html

```
package com.example.studentportal;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
public class LoginServlet extends HttpServlet {
    @Override
    protected void doPost(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        String username = req.getParameter("username").trim();
        String password = req.getParameter("password");
        if ("Srijana Subedi".equalsIgnoreCase(username) && "Srijana@123".equals(password)) {
            // Create Session
            HttpSession session = req.getSession(true);
            session.setAttribute("username", username);
            // Create Cookie
            Cookie userCookie = new Cookie("username", username);
            userCookie.setMaxAge(3600);
            resp.addCookie(userCookie);
            resp.sendRedirect("dashboard");
        } else {
            req.setAttribute("error", "Invalid username or password.");
            RequestDispatcher rd = req.getRequestDispatcher("error.jsp");
            rd.forward(req, resp);
        }
    }
}
```

DashBoardServlet.java:

```
package com.example.studentportal;
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;
public class DashboardServlet extends HttpServlet {
    @Override
    protected void doGet(HttpServletRequest req, HttpServletResponse resp) throws ServletException, IOException {
        HttpSession session = req.getSession(false);
        if (session != null && session.getAttribute("username") != null) {
            RequestDispatcher rd = req.getRequestDispatcher("dashboard.jsp");
            rd.forward(req, resp);
        } else {
            resp.sendRedirect("index.html");
        }
    }
}
```

dashboard.jsp

```
<%@ page session="true" %>
<html>
<head><title>Welcome</title></head>
<body>
  <h2>Welcome, <%= session.getAttribute("username") %>!</h2>
  <p>This content was dynamically rendered with JSP.</p>
</body>
</html>
```

error.jsp:

```
<!--error.jsp -->
<%@ page isErrorPage="true"%>
<html>
<head><title>Error</title></head>
<body>
  <h2>Login Failed</h2>
  <p><%= request.getAttribute("error") %></p>
  <a href="index.html">Tryagain</a>
</body>
</html>
```

web.xml:

```
<web-app xmlns="http://xmlns.jcp.org/xml/ns/javaee" version="3.1">
  <servlet>
    <servlet-name>LoginServlet</servlet-name>
    <servlet-class>com.example.studentportal.LoginServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>LoginServlet</servlet-name>
    <url-pattern>/login</url-pattern>
  </servlet-mapping>
  <servlet>
    <servlet-name>DashboardServlet</servlet-name>
    <servlet-class>com.example.studentportal.DashboardServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>DashboardServlet</servlet-name>
    <url-pattern>/dashboard</url-pattern>
  </servlet-mapping>
</web-app>
```



Login

Username:

Password:

Design and implement usable JavaBean class named Student Bean with the following features : Properties: Include name ,rollNumber ,and percentage as properties with standard getter and setter methods, Bound Property: Implement name as a bound property that notifies listeners of changes, Constrained Property: Implement percentage as a constrained property that validates input and allows vetoing of invalid changes, Introspection: Create a StudentBeanInfo class to customize property descriptors and demonstrate introspection, Persistence: Enable serialization to save and restore the state of the bean using Java's built-in serialization mechanism.

StudentBeans.java:

```
package com.example.bean;
import java.beans.*;
import java.io.Serializable;
public class StudentBean implements Serializable {
    private String name;
    private int rollNumber;
    private double percentage;
    private final PropertyChangeSupport pcs = new PropertyChangeSupport(this);
    private final VetoableChangeSupport vcs = new VetoableChangeSupport(this);
    public String getName() { return name; }
    public void setName(String name) {
        String oldName = this.name;
        this.name = name;
        pcs.firePropertyChange("name", oldName, name);
    }
    public int getRollNumber() { return rollNumber; }
    public void setRollNumber(int rollNumber) {
        this.rollNumber = rollNumber;
    }
    public double getPercentage() { return percentage; }
    public void setPercentage(double percentage) throws PropertyVetoException {
        double old = this.percentage;
        vcs.fireVetoableChange("percentage", old, percentage);
        this.percentage = percentage;
        pcs.firePropertyChange("percentage", old, percentage);
    }
    public void addPropertyChangeListener(PropertyChangeListener listener) {
        pcs.addPropertyChangeListener(listener);
    }
    public void addVetoableChangeListener(VetoableChangeListener listener) {
        vcs.addVetoableChangeListener(listener);
    }
}
```

StudentBeanInfo.java:

```
package com.example.bean;
import java.beans.*;
public class StudentBeanInfo extends SimpleBeanInfo {
    @Override
    public PropertyDescriptor[] getPropertyDescriptors() {
        try {
            PropertyDescriptor name = new PropertyDescriptor("name", StudentBean.class);
            PropertyDescriptor roll = new PropertyDescriptor("rollNumber", StudentBean.class);
            PropertyDescriptor percent = new PropertyDescriptor("percentage", StudentBean.class);
            return new PropertyDescriptor[]{name, roll, percent};
        } catch (IntrospectionException e) {
            e.printStackTrace();
            return null;
        }
    }
}
```

Main.java

```
package com.example.bean;
```



```

import java.beans.*;
import java.io.*;
public class Main {
    public static void main(String[] args) {
        StudentBean student = new StudentBean();
        student.addPropertyChangeListener(evt -> {
            System.out.println("[PropertyChange] " + evt.getPropertyName() +
                " changed from " + evt.getOldValue() + " to " + evt.getNewValue());
        });
        student.addVetoableChangeListener(evt -> {
            double newVal = (double) evt.getNewValue();
            if (newVal < 0 || newVal > 100) {
                throw new PropertyVetoException("Percentage must be 0-100", evt);
            }
        });
        student.setName("Srijana Subedi ");
        student.setRollNumber(3279616);
        try {
            student.setPercentage(88.5);
        } catch (PropertyVetoException e) {
            System.out.println("Vetoed: " + e.getMessage());
        }
        try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("student.ser"))) {
            oos.writeObject(student);
            System.out.println("Bean serialized successfully.");
        } catch (IOException e) { e.printStackTrace(); }
        try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("student.ser"))) {
            StudentBean loaded = (StudentBean) ois.readObject();
            System.out.println("\nDeserialized Bean Info:");
            System.out.println("Name: " + loaded.getName());
            System.out.println("Roll: " + loaded.getRollNumber());
            System.out.println("Percentage: " + loaded.getPercentage());
        } catch (IOException | ClassNotFoundException e) { e.printStackTrace(); }
    }
}

```

Output

```

[PropertyChange] name changed from null to Srijana Subedi
[PropertyChange] percentage changed from 0.0 to 88.5
Bean serialized successfully.

Deserialized Bean Info:
Name: Srijana Subedi
Roll: 3279616
Percentage: 88.5

```

Write a program to show the jdbc connection and perform crud operation.

Database code

```
CREATE DATABASE student_details;
USE student_details;
CREATE TABLE student(
id INT PRIMARY KEY,
name VARCHAR(50),
address VARCHAR(100),
email VARCHAR(50),
age INT
);
```

StudentForm.java

```
import javax.swing.*.*;
import java.awt.event.*;
import java.sql.*;

public class StudentForm extends JFrame implements ActionListener {
    JTextField t1,t2,t3,t4,t5;
    JButton insert,update,delete,display;
    Connection con;
    PreparedStatement pst;
    ResultSet rs;
    StudentForm() {
        setLayout(null);
        JLabel l1 = new JLabel("ID");
        JLabel l2 = new JLabel("Name");
        JLabel l3 = new JLabel("Address");
        JLabel l4 = new JLabel("Email");
        JLabel l5 = new JLabel("Age");
        l1.setBounds(50,50,100,30);
        l2.setBounds(50,100,100,30);
        l3.setBounds(50,150,100,30);
        l4.setBounds(50,200,100,30);
        l5.setBounds(50,250,100,30);
        add(l1); add(l2); add(l3); add(l4); add(l5);
        t1 = new JTextField();
        t2 = new JTextField();
        t3 = new JTextField();
        t4 = new JTextField();
        t5 = new JTextField();
        t1.setBounds(150,50,150,30);
        t2.setBounds(150,100,150,30);
        t3.setBounds(150,150,150,30);
        t4.setBounds(150,200,150,30);
        t5.setBounds(150,250,150,30);
        add(t1); add(t2); add(t3); add(t4); add(t5);
        insert = new JButton("Insert");
        update = new JButton("Update");
        delete = new JButton("Delete");
        display = new JButton("Display");
        insert.setBounds(20,320,80,30);
        update.setBounds(110,320,80,30);
        delete.setBounds(200,320,80,30);
        display.setBounds(290,320,90,30);
        add(insert); add(update); add(delete); add(display);
        insert.addActionListener(this);
        update.addActionListener(this);
        delete.addActionListener(this);
        display.addActionListener(this);
    }
}
```

```

setSize(420,420);
setVisible(true);
connect();
}
public void connect() {
    try {
        Class.forName("com.mysql.cj.jdbc.Driver");
        con = DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/student_details",
            "root",
            "root123"
        );
    } catch (Exception e) {
        System.out.println(e);
    }
}
public void actionPerformed(ActionEvent e) {
    try {
        if(e.getSource()==insert) {
            pst = con.prepareStatement(
                "insert into student values(?,?,?,?);");
            pst.setInt(1,Integer.parseInt(t1.getText()));
            pst.setString(2,t2.getText());
            pst.setString(3,t3.getText());
            pst.setString(4,t4.getText());
            pst.setInt(5,Integer.parseInt(t5.getText()));
            pst.executeUpdate();
            JOptionPane.showMessageDialog(this,"Inserted");
        }
        if(e.getSource()==update) {
            pst = con.prepareStatement(
                "update student set name=?,address=?,email=?,age=? where id=?");
            pst.setString(1,t2.getText());
            pst.setString(2,t3.getText());
            pst.setString(3,t4.getText());
            pst.setInt(4,Integer.parseInt(t5.getText()));
            pst.setInt(5,Integer.parseInt(t1.getText()));
            pst.executeUpdate();
            JOptionPane.showMessageDialog(this,"Updated");
        }
        if(e.getSource()==delete) {
            pst = con.prepareStatement(
                "delete from student where id=?");
            pst.setInt(1,Integer.parseInt(t1.getText()));
            pst.executeUpdate();
            JOptionPane.showMessageDialog(this,"Deleted");
        }
        if(e.getSource()==display) {

            pst = con.prepareStatement("select * from student");
            rs = pst.executeQuery();
            String data = "";
            while(rs.next()) {
                data += rs.getInt("id") + " "
                    + rs.getString("name") + " "
                    + rs.getString("address") + " "
                    + rs.getString("email") + " "
                    + rs.getInt("age") + "\n";
            }
        }
    }
}

```

```

        JOptionPane.showMessageDialog(this,data);
    }
    } catch(Exception ex) {
        System.out.println(ex);
    }
}
public static void main(String args[]) {
    new StudentForm();
}
}

```

StudentForm window showing input fields for ID, Name, Address, Email, and Age. Below the fields are three buttons: Insert, Update, and Delete.

StudentForm window with a Message dialog box showing "Inserted". The dialog box has an information icon and an OK button. The background window shows the input fields and buttons, with the Age field containing the value 22.

StudentForm window with a Message dialog box showing a list of inserted records. The dialog box has an information icon and an OK button. The background window shows the input fields and buttons, with the Age field containing the value 23.

1	Sri Bkt	12@gmail.com	23
3	Srijana Ktm	sri@gmail.com	22

StudentForm window with a Message dialog box showing "Updated". The dialog box has an information icon and an OK button. The background window shows the input fields and buttons, with the Age field containing the value 23.

StudentForm window with a Message dialog box showing "Deleted". The dialog box has an information icon and an OK button. The background window shows the input fields and buttons, with the Age field containing the value 23.

